

Handling Databases in Code

Visual Basic offers two ways to work with databases— visually and in code.

The *OleDbConnection* Class

An **OleDbConnection** object supports a connection to an OLE DB data provider. In practice, you usually use OLE DB connections with all data providers except Microsoft's SQL Server. Note that, depending on the OLE DB data provider, not all properties of an **OleDbConnection** object may be supported.

A central property of connection objects is the **ConnectionString** property, which holds a string full of attribute/value pairs that contain data needed to log on to a data provider and choose a specific database. These attribute/value pairs are specific to the data provider you're using, and make up a list of items separated by semicolons. You can either assign a connection string to the connection's **ConnectionString** property, or you can pass the connection string to the connection object's constructor, like this:

```
Dim ConnectionString As String = "Provider=SQLOLEDB.1;Integrated
" & _
    Security=SSPI;Persist Security Info=False;Initial " & _
    "Catalog=pubs;Packet Size=4096;Workstation ID=STEVE;" & _
    "Use Encryption for Data=False"
Dim Connection1 As OleDbConnection = New
OleDbConnection(ConnectionString)
```

After you've created a connection object, you can open it with the **Open** method, and assign it to the **Connection** property of a command object. Then you can use the command object with a data adapter. For example, you might assign the command object to the **SelectCommand** property of a data adapter, and you can use the data adapter's **Fill** method to execute that command and fill a dataset. When done with the connection, use its **Close** method to close it.

Here is the inheritance hierarchy of this class:

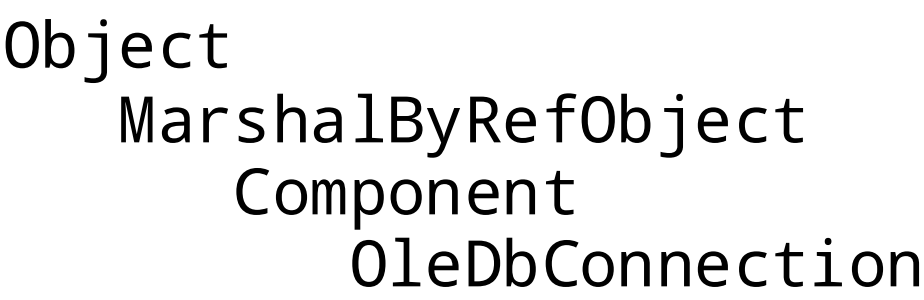


Table 22.1: Noteworthy public properties of <i>OleDbConnection</i> objects.	
Property	Means
ConnectionString	Gets/sets the connection string to open a database.
ConnectionTimeout	Gets the amount of time to wait trying to make a connection.

Table 22.1: Noteworthy public properties of <i>OleDbConnection</i> objects.	
Property	Means
Database	Gets the name of the database to open.
DataSource	Gets the data source (usually the location and file name to open).
Provider	Gets the OLE DB provider's name.
ServerVersion	Gets the version of the server.
State	Gets the connection's current state.

Table 22.2: Noteworthy public methods of <i>OleDbConnection</i> objects.	
Method	Means
BeginTransaction	Starts a database transaction.
ChangeDatabase	Changes the current database.
Close	Closes the connection to the data provider.
CreateCommand	Creates an OleDbCommand object for this connection.
GetOleDbSchemaTable	Returns the current schema table.
Open	Opens a database connection.

Table 22.3: Noteworthy public events of <i>OleDbConnection</i> objects.	
Event	Means
InfoMessage	Occurs if the provider sends a message (including warnings).
StateChange	Occurs when a connection's state changes.

The *SqlConnection* Class

A **SqlConnection** object supports a connection to a SQL Server data source. For the most part, the differences between **SqlConnection** and **OleDbConnection** objects take place behind the scenes, and the programming interface of these two types of objects is very similar.

The main difference here is one of performance-SQL connections to the Microsoft SQL Server have been shown to be up to 70 percent faster than OLE DB connections, so if you're using SQL Server, consider using SQL connections for all your connections.

The *OleDbCommand* Class

Command objects represent SQL commands or SQL stored procedures that you execute in a database. For example, to retrieve data from a database, you can create a connection object, open the connection with its **Open** method, and then assign the open connection object to the **Connection** property of a command object. You can then assign the command object to a command property of a data adapter, such as the **SelectCommand**

property (which lets you retrieve rows of data from the database when you call the data adapter's **Fill** method). Besides the **SelectCommand** property, data adapters also support **UpdateCommand**, **InsertCommand**, and **DeleteCommand** properties, each of which takes connection objects that perform these various functions.

You can either assign that text to the command object's **CommandText** property, or you can pass it to the command object's constructor For example selecting all the records in the pubs database's **authors** table:

```
Dim Command1 As OleDbCommand = _
    New OleDbCommand("SELECT * FROM authors")
```

CommandType.Text (this is the default), and assign an open connection to the command's **Connection** property:

```
Dim Command1 As OleDbCommand = _
    New OleDbCommand("SELECT * FROM authors")

Command1.CommandType = CommandType.Text

Connection1.Open()
Command1.Connection = Connection1
```

You also can use the command object's built-in methods to execute commands in a database, no data adapter needed:

- **ExecuteReader**— Executes SQL commands that return rows. Note that this method does *not* return a dataset, it creates a data reader, which is much more simplistic. See "Using a Data Reader" in the Immediate Solutions.
- **ExecuteNonQuery**— Executes commands that do not return data rows (such as **SQL INSERT, DELETE, UPDATE**, and **SET** statements).
- **ExecuteScalar**— Calculates and returns a single value, such as a sum, from a database.

Using the *OleDbCommand* Class

The **OleDbCommand** class represents a SQL statement or stored procedure that is executed in a database by an OLE DB data provider. Here is the inheritance hierarchy of this class:

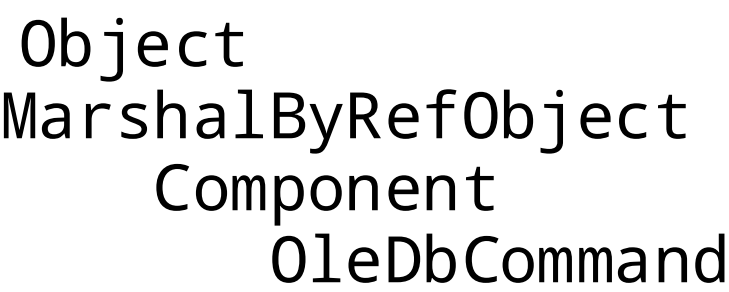


Table 22.7: Noteworthy public properties of <i>OleDbCommand</i> objects.	
Property	Means

Table 22.7: Noteworthy public properties of *OleDbCommand* objects.

Property	Means
CommandText	Gets/sets the SQL statement (or stored procedure) for this command to execute.
CommandTimeout	Gets the amount of time to wait trying to execute a command.
CommandType	Gets/sets the type of the CommandText property (typically set to text for SQL).
Connection	Gets/sets the OleDbConnection to use.
DesignTimeVisible	Gets/sets if the command object should be visible in a form designer.
Parameters	Gets the command parameters.
Transaction	Gets/sets the transaction that contains the command.
UpdatedRowSource	Gets/sets how results are used in a data row when you use the Update method.

Table 22.8: Noteworthy public methods of *OleDbCommand* objects.

Method	Means
Cancel	Cancels a command's execution.
CreateParameter	Creates a new parameter.
ExecuteNonQuery	Executes a non-row returning SQL statement, returning the number of affected rows.
ExecuteReader	Creates a data reader using the command.
ExecuteScalar	Executes the command and returns the value in the first column in the first row of the result.
Prepare	Creates a compiled version of the command.
ResetCommandTimeout	Resets the timeout value to the default value.

The *SqlCommand* Class

SqlCommand objects are very nearly the same as **OleDbCommand** objects, except they're designed to be used with SQL connections, not OLE DB connections. **SqlCommand** objects support all the methods listed in that topic, as well as one more:

ExecuteXmlReader, which creates an **XMLReader** object that makes handling the database with XML easy.

Using the *SqlCommand* Class

The **SqlCommand** class represents an SQL statement or stored procedure for use in a database using SQL Server. Here is the inheritance hierarchy of this class:

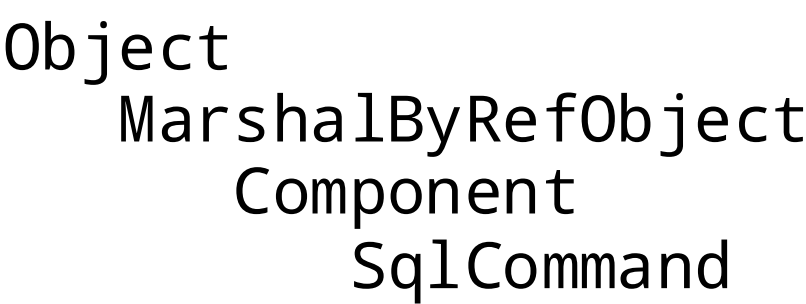


Table 22.9: Noteworthy public properties of *SqlCommand* objects.

Property	Means
CommandText	Gets/sets the SQL statement (or stored procedure) for this command to execute.
CommandTimeout	Gets the amount of time to wait trying to execute a command.
CommandType	Gets/sets the type of the CommandText property (typically set to text for SQL).
Connection	Gets/sets the SqlConnection to use.
DesignTimeVisible	Gets/sets if the command object should be visible in a form designer.
Parameters	Gets the command parameters.
Transaction	Gets/sets the transaction that contains the command.
UpdatedRowSource	Gets/sets how results are used in a data row when you use the Update method.

Table 22.10: Noteworthy public methods of *SqlCommand* objects.

Method	Means
Cancel	Cancels a command's execution.
CreateParameter	Creates a new parameter.
ExecuteNonQuery	Executes a non-row returning SQL statement, returning the number of affected rows.
ExecuteReader	Creates a data reader using the command.
ExecuteScalar	Executes the command and returns the value in the first column in the first row of the result.
ExecuteXmlReader	Builds an XmlReader object.
Prepare	Creates a compiled version of the command.
ResetCommandTimeout	Resets the timeout value to the default value.

The *OleDbDataAdapter* Class

OleDbDataAdapter objects act as a bridge between datasets and data sources. As you know, datasets are really just repositories of data; they're not directly connected to a database. **OleDbDataAdapter** objects connect datasets and data sources by supporting the **Fill** method to load data from the data source into the dataset, and the **Update** method to send changes you've made in the dataset back to the data source.

After you've created a data connection and used it to create a command object, you can assign the command object to one of the command properties of the data adapter—**SelectCommand**, **InsertCommand**, **DeleteCommand**, and **UpdateCommand**.

You also have to specify a *table mapping* when creating a data adapter object. The names of the tables you use in a dataset can be different from those in the database, depending on how you've named them, and a table mapping relates the table names in the database to the names in the dataset. For example

```
Dim Table1Mappings As New DataTableMappingCollection()  
Table1Mappings.Add("authors", "writers")  
Table1Mappings.Add("publishers", "company")
```

If you do not specify a **TableName** or a **TableMapping** name when calling the **Fill** or **Update** method of a data adapter, the data adapter searches for a **TableMapping** object named "**Table**". If it can't find that object, the data adapter uses the name "**Table**" for the data source table, and that means you can create a default table mapping by creating a **TableMapping** object using the table name "**Table**". For example, here's how I create a new **OleDbDataAdapter** object, set up the select command object it should use to populate datasets, create a default table mapping, and fill a dataset named **ds** with the **authors** table, using this adapter:

```
Dim OleDbDataAdapter1 As OleDbDataAdapter = New  
OleDbDataAdapter()  
  
OleDbDataAdapter1.SelectCommand = Command1  
OleDbDataAdapter1.TableMappings.Add("Table", "authors")  
OleDbDataAdapter1.Fill(ds)
```

Using the *DataAdapter* Class

The **DataAdapter** class is the base class for data adapters, which represent a bridge between a dataset and a database in a data provider. Here is the inheritance hierarchy of this class:



Table 22.11: Noteworthy public properties of <i>DataAdapter</i> objects.	
Property	Means
AcceptChangesDuringFill	Gets/sets if a data row's AcceptChanges method is called after it is added to a table.
MissingMappingAction	Sets the action taken if there's no table mapping for new data.
MissingSchemaAction	Sets the action taken if new data does not match an existing data schema.
TableMappings	Gets the master mapping between source tables and a data table.

Table 22.12: Noteworthy public methods of <i>DataAdapter</i> objects.	
Method	Means
Fill	Adds or updates rows in a data set to match those in the data source. Creates a table named "Table" by default.
FillSchema	Adds a table named "Table" to the specified DataSet , making the table's schema match that in the data source.
GetFillParameters	Gets the parameters to use when executing a SELECT statement in SQL.
Update	Updates the data store by calling the INSERT , UPDATE , or DELETE statements for each inserted, updated, or deleted row in the given dataset.

Using the *DbDataAdapter* Class

The **DbDataAdapter** class is the base class for the **OleDbDataAdapter** and **SqlDataAdapter** classes. Here is the inheritance hierarchy of this class:

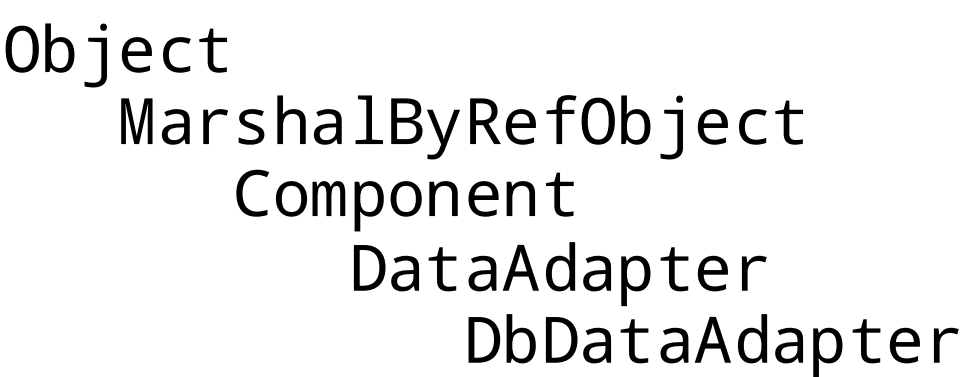


Table 22.13: Noteworthy public methods of <i>DbDataAdapter</i> objects.

Method	Means
Fill	Adds or updates rows in a data set to match those in the data source. Creates a table named "Table" by default.
FillSchema	Adds a table named "Table" to the specified dataset, making the table's schema match that in the data source.
GetFillParameters	Gets the parameters to use when executing a SELECT statement in SQL.
Update	Updates the data store by calling the INSERT , UPDATE , or DELETE statements for each inserted, updated, or deleted row in the given dataset.

Table 22.14: Noteworthy public events of *DbDataAdapter* objects.

Event	Means
FillError	Occurs when an error happens while executing a fill operation.

Using the *OleDbDataAdapter* Class

The **OleDbDataAdapter** class represents a bridge between a dataset and an OLE DB database. Here is the inheritance hierarchy of this class:

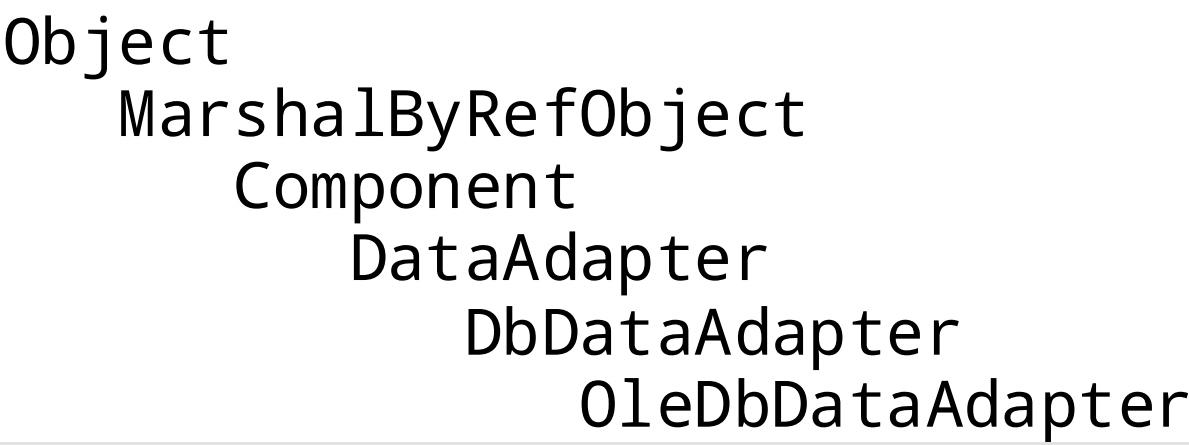


Table 22.15: Noteworthy public properties of *OleDbDataAdapter* objects.

Property	Means
DeleteCommand	Gets/sets the SQL for deleting records.
InsertCommand	Gets/sets the SQL for inserting new records.
SelectCommand	Gets/sets the SQL for selecting records.
UpdateCommand	Gets/sets the SQL for updating records.

Table 22.16: Noteworthy public methods of *OleDbDataAdapter* objects.

Method	Means
Fill	Adds or refreshes rows to a dataset to make them match the rows in a data store.

Table 22.17: Noteworthy public events of *OleDbDataAdapter* objects.

Event	Means
-------	-------

Table 22.17: Noteworthy public events of *OleDbDataAdapter* objects.

Event	Means
RowUpdated	Occurs when a row is updated.
RowUpdating	Occurs when a row is being updated.

The *SqlDataAdapter* Class

The **SqlDataAdapter** class is the SQL Server counterpart of the **OleDbDataAdapter** class. Like the **OleDbDataAdapter** class, the **SqlDataAdapter** class includes the **SelectCommand**, **InsertCommand**, **DeleteCommand**, **Update Command**, and **TableMappings** properties you use for loading and updating data.

Using the *SqlDataAdapter* Class

The **SqlDataAdapter** class represents a bridge between a dataset and an SQL Server database. Here is the inheritance hierarchy of this class:

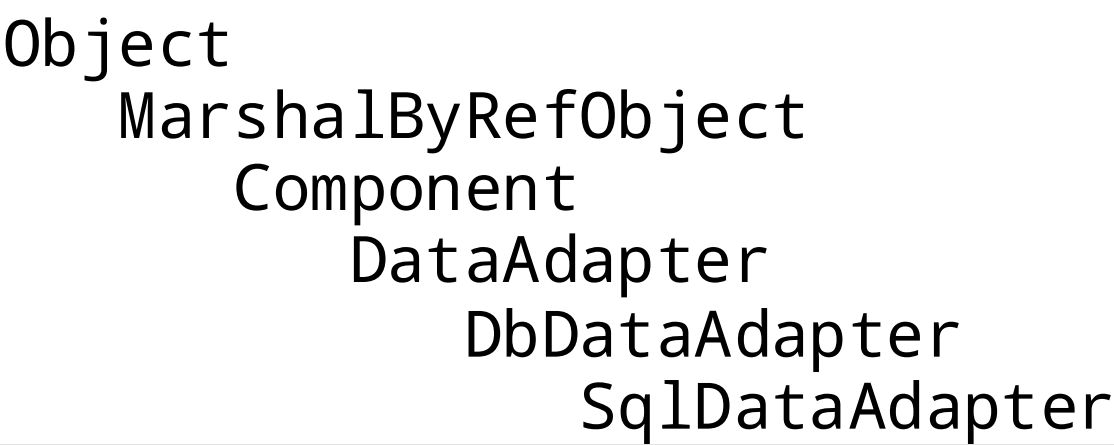


Table 22.18: Noteworthy public properties of *SqlDataAdapter* objects.

Property	Means
DeleteCommand	Gets/sets the SQL for deleting records.
InsertCommand	Gets/sets the SQL for inserting new records.
SelectCommand	Gets/sets the SQL for selecting records.
UpdateCommand	Gets/sets the SQL for updating records.

Table 22.19: Noteworthy public methods of *SqlDataAdapter* objects.

Method	Means
Fill	Adds or refreshes rows to a dataset to make them match the rows in a data store.

Table 22.20: Noteworthy public events of *SqlDataAdapter* objects.

Event	Means
RowUpdated	Occurs when a row is updated.
RowUpdating	Occurs when a row is being updated.

The *DataSet* Class

A dataset is a cache of data retrieved from a database, it's the major component of ADO.NET. A **DataSet** object is made up of a collection of **DataTable** objects that you can relate to each other using **DataRelation** objects. You also can guarantee data integrity with the **uniqueConstraint** and **ForeignKeyConstraint** objects. When the user edits data in data-bound controls changes are made to the dataset's data immediately. You can use the **GetChanges** method to get a new dataset holding only the rows that have changed, and you typically send this new dataset to the database in the data provider with the data adapter's **Update** method. The data provider may make changes itself in the dataset of changes you send it and return a new dataset, Then you use the **AcceptChanges** method on the original dataset to accept the changes (or use **RejectChanges** to cancel the changes).

Datasets can be typed or untyped; usually, datasets are typed in Visual Basic. A typed dataset is a dataset that is derived from the **DataSet** class and uses information in an XML schema file (an XSD file). An untyped dataset, on the other hand, has no built-in schema. An untyped dataset can contain tables, columns, and rows, but those are exposed only as collections.

Using the *DataSet* Class

The **DataSet** class supports datasets, which act as data caches you can access in code. Here is the inheritance hierarchy of this class:

Object
 MarshalByValueComponent
 DataSet

Table 22.21: Noteworthy public properties of *DataSet* objects.

Property	Means
CaseSensitive	Gets/sets whether string comparisons are case-sensitive.
DataSetName	Gets/sets the name of the dataset.
EnforceConstraints	Gets/sets if constraint rules are enforced.
HasErrors	Indicates if there are errors in any row of any table.
Locale	Gets/sets the locale data to compare strings.
Namespace	Gets/sets the namespace of the dataset.
Relations	Get relation objects that link tables.
Tables	Gets tables in the dataset.

Table 22.22: Noteworthy public methods of *DataSet* objects.

Method	Means
AcceptChanges	Accepts (commits) the changes made to the dataset.

Table 22.22: Noteworthy public methods of *DataSet* objects.

Method	Means
BeginInit	Begins the initialization of a dataset.
Clear	Clears the dataset by removing all rows in all tables.
Copy	Copies the dataset.
EndInit	Ends the initialization of a dataset.
GetChanges	Gets a dataset containing all changes made to the current dataset.
GetXml	Returns the data in the dataset in XML.
GetXmlSchema	Returns the schema for the dataset.
HasChanges	Indicates if the dataset has changes that have not yet been accepted.
Merge	Merges this dataset with another dataset.
ReadXml	Reads data into a dataset from XML.
ReadXmlSchema	Reads an XML schema into a dataset.
RejectChanges	Rolls back the changes made to the dataset since it was created or since the AcceptChanges method was called.
Reset	Resets the dataset back to the original state.
WriteXml	Writes the dataset's schema and data to XML.
WriteXmlSchema	Writes the dataset's schema to XML.

Table 22.23: Noteworthy public events of *DataSet* objects.

Event	Means
MergeFailed	Occurs when a Merge operation fails.

The *OleDbDataReader* Class

The **OleDbDataReader** class gives you a way of reading a *forward-only* stream of data rows from a database. Because this stream of data is "forward-only," you can read rows only one after the other, not choose any row you want, or go backward. Data readers are really low-level objects that give you direct access to the data in a database in a way that's not as structured as a dataset, but with faster access. You use this class, the **OleDbDataReader** class, with OLE DB providers.

Using the *OleDbDataReader* Class

The **OleDbDataReader** class creates a data reader for use with an OLE DB data provider. Here is the inheritance hierarchy of this class:

Object
 MarshalByRefObject

OleDbDataReader

Table 22.24: Noteworthy public properties of *OleDbDataReader* objects.

Property	Means
Depth	Gets the current row's nesting depth.
FieldCount	Gets the number of columns in the current row.
IsClosed	Indicates if a data reader is closed.
Item	Gets the value in a field.
RecordsAffected	Gets the number of rows changed, inserted, or deleted by an SQL statement.

Table 22.25: Noteworthy public methods of *OleDbDataReader* objects.

Method	Means
Close	Closes the data reader.
GetBoolean	Gets a field's value as a Boolean.
GetByte	Gets a field's value as a byte.
GetBytes	Reads a stream of bytes.
GetChar	Gets a field's value as a character.
GetChars	Reads a stream of characters.
GetDataTypeName	Gets the name of the source data type.
GetDateTime	Gets a field's value as a DateTime object.
GetDecimal	Gets a field's value as a Decimal object.
GetDouble	Gets a field's value as a double-precision floating point number.
GetFieldType	Gets the Type that is the data type of the object.
GetFloat	Gets a field's value as a single-precision floating point number.
GetGuid	Gets a field's value as a globally unique identifier (GUID).
GetInt16	Gets a field's value as a 16-bit signed integer.
GetInt32	Gets a field's value as a 32-bit signed integer.
GetInt64	Gets a field's value as a 64-bit signed integer.
GetName	Gets the name of the specified column.
GetOrdinal	Gets the column ordinal, given the name of the column.
GetSchemaTable	Returns a schema.
GetString	Gets a field's value as a string.
GetValue	Gets the value of the column in its original format.
GetValues	Gets all the attribute columns in the current row.
IsDBNull	Indicates if a column contains nonexistent (or missing) values.

Table 22.25: Noteworthy public methods of *OleDbDataReader* objects.

Method	Means
Read	Advances a data reader to the next record and reads that record.

The *SqlDataReader* Class

The **SqlDataReader** class is the SQL Server version of the **OleDbDataReader** class. There are very few difference between the **SqlDataReader** class and the **OleDbDataReader** class; for most practical purposes, you use them the same way-you use the **Read** method to read a new row from a database.

The **SqlDataReader** class creates a data reader for use with the SQL Server. Here is the inheritance hierarchy of this class:

Object
 MarshalByRefObject
 SqlDataReader

Table 22.26: Noteworthy public properties of *SqlDataReader* objects.

Property	Means
Depth	Gets the current row's nesting depth.
FieldCount	Gets the number of columns in the current row.
IsClosed	Indicates if a data reader is closed.
Item	Gets the value in a field.
RecordsAffected	Gets the number of rows changed, inserted, or deleted by an SQL statement.

Table 22.27: Noteworthy public methods of *SqlDataReader* objects.

Method	Means
Close	Closes the data reader.
GetBoolean	Gets a field's value as a Boolean.
GetByte	Gets a field's value as a byte.
GetBytes	Reads a stream of bytes.
GetChar	Gets a field's value as a single character.
GetChars	Reads a stream of characters.
GetData	Not currently supported.
GetDataTypeName	Gets the name of the source data type.
GetDateTime	Gets a field's value as a DateTime object.
GetDecimal	Gets a field's value as a Decimal object.

Table 22.27: Noteworthy public methods of *SqlDataReader* objects.

Method	Means
GetDouble	Gets a field's value as a double-precision floating point number.
GetFieldType	Gets the Type that is the data type of the object.
GetFloat	Gets a field's value as a single-precision floating point number.
GetGuid	Gets a field's value as a globally unique identifier (GUID).
GetInt16	Gets a field's value as a 16-bit signed integer.
GetInt32	Gets a field's value as a 32-bit signed integer.
GetInt64	Gets a field's value as a 64-bit signed integer.
GetName	Gets the name of the specified column.
GetOrdinal	Gets the column ordinal, given the name of the column.
GetSchemaTable	Returns a schema.
GetSqlBinary	Gets a field's value as a SqlBinary .
GetSqlByte	Gets a field's value as a SqlByte .
GetSqlDateTime	Gets a field's value as a SqlDateTime .
GetSqlDecimal	Gets a field's value as a SqlDecimal .
GetSqlDouble	Gets a field's value as a SqlDouble .
GetSqlGuid	Gets a field's value as a SqlGuid .
GetSqlInt16	Gets a field's value as a SqlInt16 .
GetSqlInt32	Gets a field's value as a SqlInt32 .
GetSqlInt64	Gets a field's value as a SqlInt64 .
GetSqlMoney	Gets a field's value as a SqlMoney .
GetSqlSingle	Gets a field's value as a SqlSingle .
GetSqlString	Gets a field's value as a SqlString .
GetSqlValue	Gets an object of SqlDbType variant.
GetSqlValues	Gets all the attribute columns in the current row.
GetString	Gets a field's value as a string.
GetValue	Gets a field's value in its native format.
GetValues	Gets all attribute columns in the collection for the current row.
IsDBNull	Indicates if a column contains nonexistent (or missing) values.
Read	Advances a data reader to the next record and reads that record.

The *DataTable* Class

The **DataTable** class represents a table of data. **DataTable** objects store data tables, and as such, they're central to datasets and data views. In code, you create data tables and then add the fields in each row to them. Note that after creating the table, create a new **DataColumn** object, configure it, and add it to the table's **Columns** collection:

```
Dim Table1 As DataTable
Table1 = New DataTable("Employees")
Dim FirstName As DataColumn = New DataColumn("First Name")
FirstName.DataType = System.Type.GetType("System.String")
Table1.Columns.Add(FirstName)
```

To add rows to a **DataTable**, you use the **NewRow** method to return a new **DataRow** object, because the **NewRow** method returns a row with the schema of the **DataTable**. Then you add data to the fields in the row, referring to them by name or index, and add the row back to the table's **Rows** collection. Example:

```
Row1 = Table1.NewRow()
```

```
Row1("First Name") = "Ralph"
Row1("Last Name") = "Kramden"
Row1("ID") = 1
Row1("Phone") = "(555) 111-2222"
```

```
Table1.Rows.Add(Row1)
```

You can add it to a dataset, and bind that dataset to the data grid, like this:

```
Dim ds As New DataSet()
ds = New DataSet()
ds.Tables.Add(Table1)
DataGrid1.SetDataBinding(ds, "Employees")
```

The *DataRow* Class

DataRow objects represent rows in a **DataTable** object. You use **DataRow** objects to get access to, insert, delete, and update the records in a table.

To create a new **DataRow** object, you usually use the **NewRow** method of a **DataTable** object, and after configuring the row with data, you can use the **Add** method to add the new **DataRow** to the table. In addition, you also can call the **AcceptChanges** method of the **DataTable** object to make that table treat the new row as it would its original data.

You can delete a **DataRow** from the **Rows** collection in a data table by calling the **Remove** method, or by calling the **Delete** method of the **DataRow** object itself. Note that the **Remove** removes the row from the collection, and the **Delete** method simply marks the

DataRow for deletion. (The actual deletion occurs when you use the **AcceptChanges** method.)

```
For RowLoopIndex = 0 To (DataSet11.Tables("authors").Rows.Count - 1)
    For ColLoopIndex = 0 To (DataSet11.Tables("authors").Columns.Count - 1)
        TextBox1.Text &= DataSet11.Tables("authors").Rows(RowLoopIndex).Item(ColLoopIndex) & _
            ControlChars.Tab & ControlChars.Tab
    Next ColLoopIndex
    TextBox1.Text &= ControlChars.CrLf
Next RowLoopIndex
```

Using the *DataRow* Class

The **DataRow** class represents a data row of data in a data table. Here is the inheritance hierarchy of this class:

Object
 DataRow

Table 22.31: Noteworthy public properties of <i>DataRow</i> objects.	
Property	Means
HasErrors	Indicates if there are errors in the row.
Item	Gets/sets data in a specified column.
ItemArray	Gets/sets all the data in a row.
RowError	Gets/sets a row's error description.
RowState	Gets the current state of a row.
Table	Gets the table that contains this row.

Table 22.32: Noteworthy public methods of <i>DataRow</i> objects.	
Method	Means
AcceptChanges	Accepts (commits) the changes made to the row.
BeginEdit	Begins an edit operation.
CancelEdit	Cancels the current edit operation.
ClearErrors	Clears the errors in the row.
Delete	Deletes the row.

Table 22.32: Noteworthy public methods of *DataRow* objects.

Method	Means
EndEdit	Ends the current edit operation.
GetChildRows	Gets the row's child rows.
GetColumnError	Gets a column's error description.
GetColumnsnError	Gets the columns that have errors.
GetParentRow	Gets the parent row of a row.
GetParentRows	Gets the parent rows of a row.
IsNull	Indicates if a column contains a null value.
RejectChanges	Rolls back the changes made to the table since it was created or since the AcceptChanges method was called.
SetColumnError	Sets a column's error description.
SetParentRow	Sets the parent row of a row.

The *DataColumn* Class

DataColumn objects represent the columns, that is, the fields, in a data table. In ADO.NET terms, the columns in a table specify its XML schema. When you create a table and add columns to it, you specify the name of the column and the type of data it stores.

Using the *DataColumn* Class

Here is the inheritance hierarchy of this class:

Object
 MarshalByValueComponent
 DataColumn

Table 22.33: Noteworthy public properties of *DataColumn* objects.

Property	Means
AllowDBNull	Gets/sets if null values are allowed.
AutoIncrement	Gets/sets if the column automatically increments the column's value

nt	when new rows are added to the table.
AutoIncrementSeed	Gets/sets the starting value for an autoincrement column.
AutoIncrementStep	Gets/sets the increment for an autoincrement column.
Caption	Gets/sets the caption for the column.
ColumnMapping	Gets/sets the column's mapping type.
ColumnName	Gets/sets the name of the column.
DataType	Gets/sets the type of data in the column.
DefaultValue	Gets/sets the default value for the column (used in new rows).
Expression	Gets/sets an expression used to filter rows, calculate the values, create aggregate values, and so on.
MaxLength	Gets/sets the maximum length of a text column.
Namespace	Gets/sets the XML namespace of the column.
Ordinal	Gets the position of the column in the Columns collection.
ReadOnly	Gets/sets if the column is read-only.
Table	Gets the table the column belongs to.
Unique	Gets/sets if the values in this column must be unique.

Table 22.34: Noteworthy public methods of *DataColumn* objects.

Table 22.34: Noteworthy public methods of <i>DataColumn</i> objects.	
Method	Means
ToString	Gets the Expression value for this column, if there is one.

The *DataRelation* Class

DataRelation objects relate two data table objects to each other through the use of DataColumn objects. Datasets are just simple data repositories, and when you load them, they don't know anything about the relations between tables in a relational database.

Using the *DataRelation* Class

Here is the inheritance hierarchy of this class:



Table 22.35: Noteworthy public properties of *DataRelation* objects.

Property	Means
ChildColumns	Gets the child column objects for the relation.
ChildKeyConstraint	Gets the child key constraint for the relation.
ChildTable	Gets the child table of this relation.
DataSet	Gets the dataset the relation is contained in.
Nested	Gets/sets if relations are nested.
ParentColumns	Gets the parent column objects for the relation.
ParentKeyConstraint	Gets the constraint that ensures values in the parent column of the relation are unique.
ParentTable	Gets the parent table for the relation.
RelationName	Gets/sets the name of the relation.

Table 22.36: Noteworthy public methods of *DataRelation* objects.

Method	Means
ToString	Gets the relation's name, if there is one.

Creating a Data Connection in Code

```
Private Sub Button1_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button1.Click
    Dim ds As New DataSet()
    ds = New DataSet("authors")
    :
```

Now we'll need a connection object to connect to the **authors** table, and create that connection like this, using a connection string.

```
Private Sub Button1_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button1.Click
    Dim ds As New DataSet()
    ds = New DataSet("authors")

    Dim connectionString As String =
    "Provider=SQLOLEDB.1;Integrated " & _
    "Security=SSPI;Persist Security Info=False;Initial " & _
    "Catalog=pubs;Packet Size=4096;Workstation ID=STEVE;" & _
    "Use Encryption for Data=False"

    Dim Connection1 As OleDbConnection = New _
    OleDbConnection(connectionString)
    :
```


Creating a Command Object in Code

After opening the connection object we created in the previous ,assign that connection object to the command object's **Connection** property:

```
Private Sub Button1_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button1.Click
    Dim ds As New DataSet()
    ds = New DataSet("authors")

    Dim ConnectionString As String =
"Provider=SQLOLEDB.1;Integrated " & _
    "Security=SSPI;Persist Security Info=False;Initial " & _
    "Catalog=pubs;Packet Size=4096;Workstation ID=STEVE;" & _
    "Use Encryption for Data=False"

    Dim Connection1 As OleDbConnection = New _
        OleDbConnection(ConnectionString)

    Dim Command1 As OleDbCommand = _
        New OleDbCommand("SELECT * FROM authors")
    Command1.CommandType = CommandType.Text

    Connection1.Open()
    Command1.Connection = Connection1
    :
```

Our command object is now ready to be used with a data adapter to get the **authors** table from the pubs database.

Creating a Data Adapter in Code

```
Private Sub Button1_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button1.Click
    Dim ds As New DataSet()
    ds = New DataSet("authors")

    Dim ConnectionString As String =
"Provider=SQLOLEDB.1;Integrated " & _
    "Security=SSPI;Persist Security Info=False;Initial " & _
    "Catalog=pubs;Packet Size=4096;Workstation ID=STEVE;" & _
    "Use Encryption for Data=False"

    Dim Connection1 As OleDbConnection = New _
        OleDbConnection(ConnectionString)

    Dim Command1 As OleDbCommand = _
        New OleDbCommand("SELECT * FROM authors")
    Command1.CommandType = CommandType.Text
```

```

        Connection1.Open()
        Command1.Connection = Connection1

        Dim OleDbDataAdapter1 As OleDbDataAdapter = New
OleDbDataAdapter()

        OleDbDataAdapter1.SelectCommand = Command1
        OleDbDataAdapter1.TableMappings.Add("Table", "authors")
        OleDbDataAdapter1.Fill(ds)

        DataGridView1.SetDataBinding(ds, "authors")
End Sub

```

Creating a Data Table in Code

This example will show us how to create not only a data table from scratch, but also an entire dataset.

Let's see how this works in code.

```

Private Sub Button1_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button1.Click
    Dim Table1 As DataTable
    Table1 = New DataTable("Employees")
    :

```

The next step is to stock the data table with columns,

Creating Data Columns in Code

After creating a table in the DataTableCode, it's time to create the columns in that table. The columns specify the structure of the table because they specify the type and name of each column; after the table has been so constructed, we can add data-that is, the rows.

The data types you can use for columns in Visual Basic are **System.Boolean**, **System.Byte**, **System.Char**, **System.DateTime**, **System.Decimal**, **System.Double**, **System.Int16**, **System.Int32**, **System.Int64**, **System.SByte**, **System.Single**, and **System.String**.

```

Private Sub Button1_Click(ByVal sender As System.Object, ByVal e
As_
    System.EventArgs) Handles Button1.Click
    Dim Table1 As DataTable
    Table1 = New DataTable("Employees")

    Dim FirstName As DataColumn = New DataColumn("First Name")

```

```
FirstName.DataType = System.Type.GetType("System.String")
Table1.Columns.Add(FirstName)
```

```
Dim LastName As DataColumn = New DataColumn("Last Name")
LastName.DataType = System.Type.GetType("System.String")
Table1.Columns.Add(LastName)
```

```
Dim ID As DataColumn = New DataColumn("ID")
ID.DataType = System.Type.GetType("System.Int32")
Table1.Columns.Add(ID)
```

```
Dim Phone As DataColumn = New DataColumn("Phone")
Phone.DataType = System.Type.GetType("System.String")
Table1.Columns.Add(Phone)
```

⋮

Now we've added four columns to the Employees table.

Creating Data Rows in Code

To create a **DataRow** object for insertion in a particular table, you call the table's **NewRow** method, which returns a **DataRow** object configured with the columns you've already set up in the table. You can then reach the fields in the row with the **Item** property.

```
Private Sub Button1_Click(ByVal sender As System.Object, ByVal e
As_
```

```
System.EventArgs) Handles Button1.Click
```

```
Dim Table1 As DataTable
```

```
Dim Row1, Row2, Row3, Row4 As DataRow
```

```
Table1 = New DataTable("Employees")
```

```
Dim FirstName As DataColumn = New DataColumn("First Name")
```

```
FirstName.DataType = System.Type.GetType("System.String")
```

```
Table1.Columns.Add(FirstName)
```

⋮

```
Dim Phone As DataColumn = New DataColumn("Phone")
```

```
Phone.DataType = System.Type.GetType("System.String")
```

```
Table1.Columns.Add(Phone)
```

```
Row1 = Table1.NewRow()
```

```
Row1("First Name") = "Ralph"
```

```
Row1("Last Name") = "Kramden"
```

```
Row1("ID") = 1
```

```
Row1("Phone") = "(555) 111-2222"
```

```
Table1.Rows.Add(Row1)
```

```
Row2 = Table1.NewRow()
```



```

Row2("First Name") = "Ed"
Row2("Last Name") = "Norton"
Row2("ID") = 2
Row2("Phone") = "(555) 111-3333"

Table1.Rows.Add(Row2)

Row3 = Table1.NewRow()

Row3("First Name") = "Alice"
Row3("Last Name") = "Kramden"
Row3("ID") = 3
Row3("Phone") = "(555) 111-2222"
Table1.Rows.Add(Row3)

Row4 = Table1.NewRow()

Row4("First Name") = "Trixie"
Row4("Last Name") = "Norton"
Row4("ID") = 4
Row4("Phone") = "(555) 111-3333"

Table1.Rows.Add(Row4)

Dim ds As New DataSet()
ds = New DataSet()
ds.Tables.Add(Table1)
DataGrid1.SetDataBinding(ds, "Employees")
End Sub

```

And that's all it takes; note that at the end of the code, add the new table to a dataset and bind that dataset to the data grid. In this way, we've created an entire dataset from scratch, no database connection needed.

Accessing Individual Data Items

How do you access the individual values in a database table?

To make this example work, I've dragged a data adapter onto the main form, connected it to the **authors** table, and created a dataset, **DataSet11**, from that data adapter. The first step in using this dataset to create the display to determine the name of each column to create the headers.

```

Private Sub Form1_Load(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles MyBase.Load
    Dim RowLoopIndex, ColLoopIndex As Integer
    DataSet11.Clear()
    OleDbDataAdapter1.Fill(DataSet11)

```

```

        For ColLoopIndex = 0 To
(DataSet11.Tables("authors").Columns.Count - 1)
            TextBox1.Text &= _
                DataSet11.Tables( _
                    "authors").Columns(ColLoopIndex).ColumnName & _
                    ControlChars.Tab & ControlChars.Tab
            If ColLoopIndex = 0 Then 'Handle wide au_id field
                TextBox1.Text &= ControlChars.Tab
            End If
        Next ColLoopIndex

        TextBox1.Text &= ControlChars.CrLf

        For ColLoopIndex = 0 To
(DataSet11.Tables("authors").Columns.Count - 1)
            TextBox1.Text &= "-----" _
                & ControlChars.Tab & ControlChars.Tab
            If ColLoopIndex = 0 Then 'Handle wide au_id field
                TextBox1.Text &= ControlChars.Tab
            End If
        Next ColLoopIndex

        TextBox1.Text &= ControlChars.CrLf
        :

```

Now we can get the actual data in each row of the table using the table's **Rows** collection, and the **Item** property of each **DataRow** object to get the data in each field. For example, to get the data in the first field of the first row in the **Rows** collection, we can use the expression **Rows(0).Item(0)**.

```

Private Sub Form1_Load(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles MyBase.Load
    Dim RowLoopIndex, ColLoopIndex As Integer
    DataSet11.Clear()
    OleDbDataAdapter1.Fill(DataSet11)

    For ColLoopIndex = 0 To
        (DataSet11.Tables("authors").Columns.Count - 1)
        :
    Next ColLoopIndex

    TextBox1.Text &= ControlChars.CrLf

    For ColLoopIndex = 0 To
        (DataSet11.Tables("authors").Columns.Count - 1)
        :
    Next ColLoopIndex

    TextBox1.Text &= ControlChars.CrLf

```



```

        For RowLoopIndex = 0 To
(DataSet11.Tables("authors").Rows.Count - 1)
            For ColLoopIndex = 0 To _
                (DataSet11.Tables("authors").Columns.Count - 1)
                TextBox1.Text &=
DataSet11.Tables("authors").Rows(RowLoopIndex).Item(ColLoopIndex)
& ControlChars.Tab & ControlChars.Tab
            Next ColLoopIndex
            TextBox1.Text &= ControlChars.CrLf
        Next RowLoopIndex
End Sub

```

If you prefer, you can loop over all data with **For Each** loops instead, which makes the code easier:

```

Dim CurrentRow As DataRow
Dim CurrentColumn As DataColumn

For Each CurrentRow In DataSet11.Tables("authors").Rows
    For Each CurrentColumn In DataSet11.Tables("authors").Columns
        TextBox1.Text &= CurrentRow(CurrentColumn) & ControlChars.
Tab _
            & ControlChars.Tab
    Next CurrentColumn
    TextBox1.Text &= ControlChars.CrLf
Next CurrentRow

```

Note that you can access the data in the various fields of a data row by numeric index or by name, and you can abbreviate expressions by eliminating the keyword **Item**. For example, if you're working with a row of data named **CurrentRow**, and the "First Name" field is the first field in the row, all these statements are equivalent:

```

FirstName = CurrentRow.Item("First Name")
FirstName = CurrentRow("First Name")
FirstName = CurrentRow.Item(0)
FirstName = CurrentRow(0)

```

Looping over all Tables in a Dataset

if you wanted to loop over all the tables in a dataset as well? we can access the tables in a dataset with its **Tables** property, so here's how to display all the data in an entire dataset, table-by-table, row-by-row, and column-by-column:

```

For Each CurrentTable in DataSet11.Tables
    For Each CurrentRow In CurrentTable.Rows
        For Each CurrentColumn In CurrentTable.Columns
            TextBox1.Text &= CurrentRow(CurrentColumn) &
ControlChars.Tab _

```

```

        & ControlChars.Tab
    Next CurrentColumn
    TextBox1.Text &= ControlChars.CrLf
Next CurrentRow
Next CurrentTable

```

Using a Data Reader

We use data readers to get low-level access to the data in a database. Data readers let you read record after record (going forward in the database only) and retrieve individual values in each record.

There are no data reader controls in the Visual Basic toolbox— you have to create them in code. To create a data reader, you can use the **ExecuteReader** method of a command object.

```

Private Sub Button1_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button1.Click
    Dim Connection1String As New String(_
        "Provider=SQLOLEDB;Data Source=;User ID=sa;Initial
Catalog=pubs;")
    Dim Connection1 As New OleDbConnection(Connection1String)

    Dim Command1 As New OleDbCommand("SELECT * FROM authors",
Connection1)

    Connection1.Open()

    Dim Reader1 As OleDbDataReader = _
        Command1.ExecuteReader(CommandBehavior.CloseConnection)
    :
Private Sub Button1_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button1.Click
    Dim LoopIndex As Integer
    Dim Connection1String As New String(_
        "Provider=SQLOLEDB;Data Source=;User ID=sa;Initial
Catalog=pubs;")
    :
    Dim schemaTable As DataTable = Reader1.GetSchemaTable()

    For LoopIndex = 0 To schemaTable.Rows.Count - 1
        TextBox1.Text &=
schemaTable.Rows(LoopIndex).Item(0).ToString() & _
        ControlChars.Tab & ControlChars.Tab
        If LoopIndex = 0 Then 'Handle wide au_id field
            TextBox1.Text &= ControlChars.Tab
        End If
    Next

```

```
TextBox1.Text &= ControlChars.CrLf
```

```
For LoopIndex = 0 To (schemaTable.Rows.Count - 1)
    TextBox1.Text &= "-----" & ControlChars.Tab & _
        ControlChars.Tab
    If LoopIndex = 0 Then 'Handle wide au_id field
        TextBox1.Text &= ControlChars.Tab
    End If
Next LoopIndex
```

```
TextBox1.Text &= ControlChars.CrLf
```

```
⋮
```

To actually read a row of data from a data reader, you use the **Read** method. After the reader has read a row, you use **Get** methods to read the data in the fields in the row, one after the other. If you know the field holds string data, you use the **GetString** method. If you know it holds a **Double** value, use **GetDouble**.

```
Private Sub Button1_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button1.Click
    Dim LoopIndex As Integer
    Dim Connection1String As New String(_
        "Provider=SQLOLEDB;Data Source=;User ID=sa;Initial
Catalog=pubs;")
    Dim Connection1 As New OleDbConnection(Connection1String)
    ⋮
    TextBox1.Text &= ControlChars.CrLf

    While Reader1.Read()
        For LoopIndex = 0 To schemaTable.Rows.Count - 1
            If schemaTable.Rows(LoopIndex).Item(5).ToString() = _
                "System.String" Then
                TextBox1.Text &= Reader1.GetString(LoopIndex) & _
                    ControlChars.Tab & ControlChars.Tab
            End If
            If schemaTable.Rows(LoopIndex).Item(5).ToString() = _
                "System.Boolean" Then
                TextBox1.Text &=
Reader1.GetBoolean(LoopIndex).ToString() & _
                    ControlChars.Tab & ControlChars.Tab
            End If
        Next LoopIndex
        TextBox1.Text &= ControlChars.CrLf
    End While
    Reader1.Close()
    Connection1.Close()
End Sub
```

Creating a Data Relation Object in Code

To create the new **DataRelation** object, we just pass this object's constructor the name of the new object, the parent column, and the child column. Then we add the new relation object to a dataset's **Relations** collection, and bind a data grid to that object, like this:

```
Private Sub Form1_Load(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles MyBase.Load
    DataSet11.Clear()
    OleDbDataAdapter1.Fill(DataSet11)
    OleDbDataAdapter2.Fill(DataSet11)
    Dim PublishersColumn As DataColumn
    Dim TitlesColumn As DataColumn
    PublishersColumn =
DataSet11.Tables("publishers").Columns("pub_id")
    TitlesColumn = DataSet11.Tables("titles").Columns("pub_id")

    Dim publisherstitles As DataRelation
    publisherstitles = New DataRelation("publisherstitles", _
        PublishersColumn, TitlesColumn)

    DataSet11.Relations.Add(publisherstitles)
    DataGrid1.SetDataBinding(DataSet11,
"publishers.publisherstitles")
End Sub
```

Creating a Data Relation Object in Code

To create the new **DataRelation** object, we just pass this object's constructor the name of the new object, the parent column, and the child column. Then add the new relation object to a dataset's **Relations** collection, and bind a data grid to that object:

```
Private Sub Form1_Load(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles MyBase.Load
    DataSet11.Clear()
    OleDbDataAdapter1.Fill(DataSet11)
    OleDbDataAdapter2.Fill(DataSet11)
    Dim PublishersColumn As DataColumn
    Dim TitlesColumn As DataColumn
    PublishersColumn =
DataSet11.Tables("publishers").Columns("pub_id")
    TitlesColumn = DataSet11.Tables("titles").Columns("pub_id")

    Dim publisherstitles As DataRelation
    publisherstitles = New DataRelation("publisherstitles", _
        PublishersColumn, TitlesColumn)

    DataSet11.Relations.Add(publisherstitles)
    DataGrid1.SetDataBinding(DataSet11,
"publishers.publisherstitles")
End Sub
```

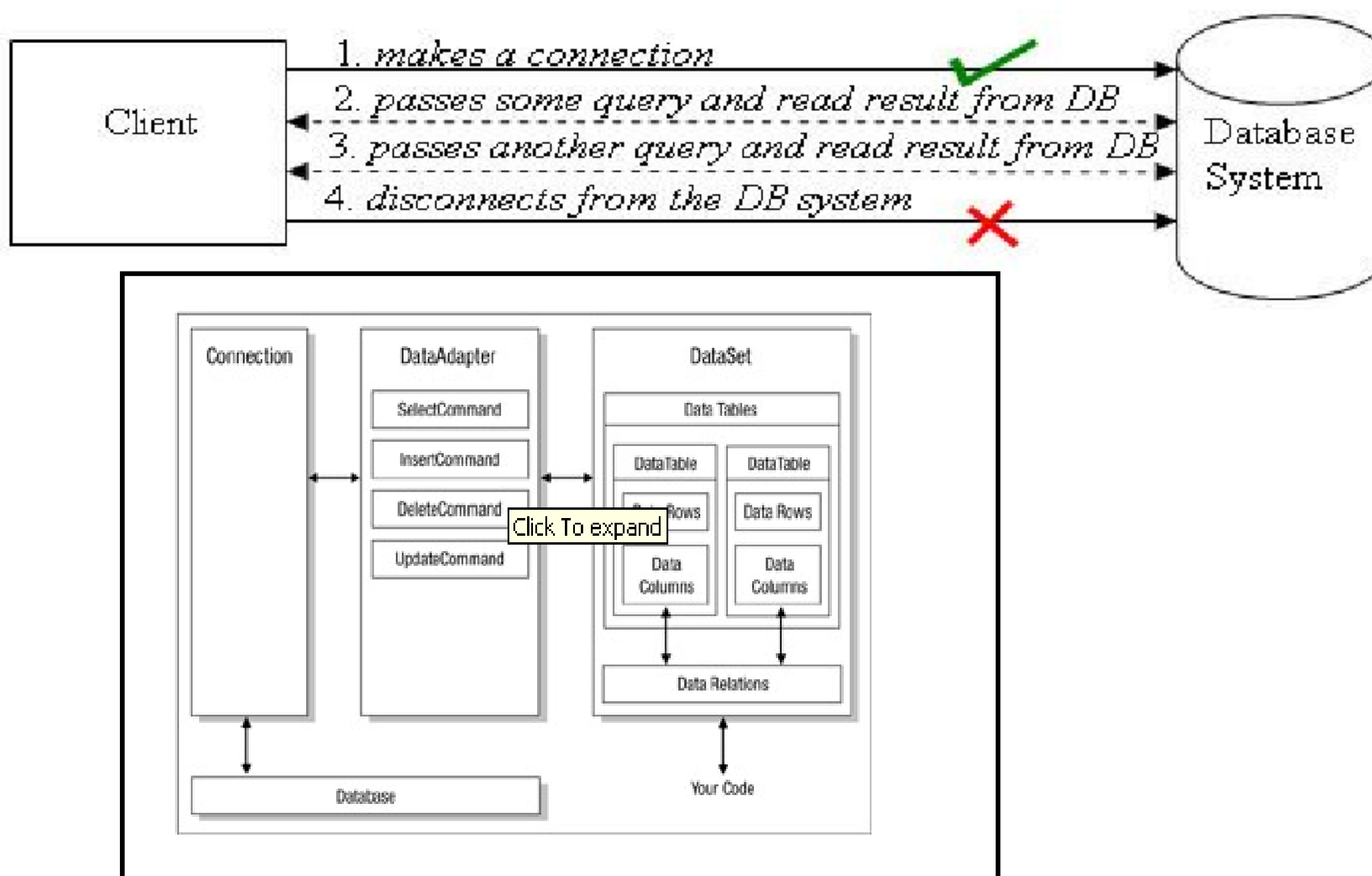
Data Access with ADO.NET

Connections, Data Adapters, and Datasets

Visual Basic .NET uses ADO.NET (ADO stands for ActiveX Data Objects) as its primary data access and manipulation protocol. a *connection* to a data source, which means using a *data provider* to access a database. The default data provider that Visual Basic .NET works with is Microsoft's SQL Server, version 7.0 or later. Visual Basic also can work with any data provider that can support Open Database Connectivity (ODBC), such as Oracle. To work with SQL server, you use ADO.NET **SqlConnection** objects, and to work with any other data provider, you use ADO.NET **OleDbConnection** objects.

After you have a connection to a data source, you create a *data adapter* to work with that data. You need a data adapter because datasets do not maintain any active connection to the database— they are *disconnected* from the database. The data adapter is what actually applies your SQL statements to a database and causes your datasets to fill with data. To work with the SQL Server data provider, you use **SQLDataAdapter** objects, and to work with ODBC data providers, you use **OleDbAdapter** objects.

Once you have a data adapter, you can generate a *dataset* using that adapter. Datasets are what you actually work within your code when you want to use data from databases. (for example, if the table was named **students**, that SQL might be "SELECT * FROM students") and then fill a **DataSet** object using that adapter. Note that each data adapter can handle only one SQL query at a time, but **DataSet** objects can store multiple tables



Accessing Data with the Server Explorer

To work with a database, you need a connection to that database. In Visual Basic, the Server Explorer lets you work with connections to various data sources. This tool lets you create and examine data connections.

In summary, here are the steps:

1. Create a data connection or use an existing data connection.
2. Drag a table from the Server Explorer onto a form; this adds a data connection and a data adapter to the form.
3. Generate a dataset.
4. Bind the dataset to controls.
5. Fill the dataset from a data adapter in code.

Accessing Data with Data Adaptors and Datasets

That creates the data adapter we'll need, **OleDbDataAdapter1**. Now create a new dataset using this data adapter and connect the new dataset to a data grid using the **DataSource** and **DataMember** properties.

```
Private Sub btnLoad_Click(ByVal sender As System.Object, _  
    ByVal e As System.EventArgs) Handles btnLoad.Click  
    DataSet11.Clear()  
    OleDbDataAdapter1.Fill(DataSet11)  
End Sub
```

The user can edit the data in a data grid directly, and update the database with it if you want to allow such operations.

```
Private Sub btnUpdate_Click(ByVal sender As System.Object, ByVal  
    e As System.EventArgs) Handles btnUpdate.Click  
  
    OleDbDataAdapter1.Update(DataSet11)  
End Sub
```

Here's a summary of the steps:

1. Create a data connection or use an existing data connection.
2. Drag an **OleDbAdaptor** or **SQLAdaptor** object onto a form; creates connection and adaptor objects.

3. Use the Data Adapter Configuration Wizard to configure the data adapter and create the SQL you want.
4. Generate a dataset.
5. Bind the dataset to controls.
6. Fill the dataset in code.

Working with ADO.NET

ADO.NET is the main data access system and protocol that Visual Basic .NET uses. As we've already mentioned, it uses a disconnected data architecture, which means that the data you work with is just a copy of the data in the data in the actual database.

To store the data you work with in your application, you use datasets, which represent a sort of data cache of records. Datasets are really just passive containers for data. To actually get data from a database and write it back, you use data adapters. A data adapter contains the instructions for populating a single table in the dataset and updating the corresponding table in the database. The instructions are methods that encapsulate either SQL commands or references to stored procedures.

Note that because a dataset exists only locally, it does not necessarily reflect the current state of the database. Although a dataset acts as a cache for data drawn from a database, the dataset has no actual direct relationship with the database. The dataset is a container; it is filled by SQL commands or stored procedures executed from a data adapter. If you want to see the latest changes made by other users, you can refresh the dataset by calling the **Fill** method.

Overview of ADO.NET Objects

Here's a list of the most common ADO.NET objects:

- *Data connection objects*— To start working with a database, you must have a data connection. A data adapter needs a connection to a data source to read and write data, and it uses **OleDbConnection** or **SqlConnection** objects to communicate with a data source.
- *Data adapters*— Data adapters are a very important part of ADO.NET. You use them to communicate between a data source and a dataset. You typically configure a data adapter with SQL to execute against the data source. The two types of data adapters are **OleDbDataAdapter** and **SqlDataAdapter** objects.
- *Command objects*— Data adapters can read, add, update, and delete records in a data source. To allow you to specify how each of these operations work, a data adapter contains command objects for each of them. Data adapters support four

properties that give you access to these command objects: **SelectCommand**, **InsertCommand**, **UpdateCommand**, and **DeleteCommand**.

- *Datasets*— Datasets store data in a disconnected cache. The structure of a dataset is similar to that of a relational database; it gives you access to an object model of tables, rows, and columns, and it contains constraints and relationships defined for the dataset. Datasets are supported with **DataSet** objects.
- **DataTable** objects— **DataTable** objects hold a data table from a data source. Data tables contain two important properties: **Columns**, which is a collection of the **DataColumn** objects that represent the columns of data in a table, and **Rows**, which is a collection of **DataRow** objects, representing the rows of data in a table.
- *Data readers*— **DataReader** objects hold a read-only, forward-only (i.e., you can only move from one record to the succeeding record, not backwards) set of data from a database. Using a data reader can increase speed because only one row of data is in memory at a time.
- *Data views*— Data views represent a customized view of a single table that can be filtered, searched, or sorted. In other words, a data view, supported by the **DataView** class, is a data "snapshot" that takes up few resources.
- *Constraint objects*— Datasets support constraints to check data integrity. A constraint, supported by the **Constraint** class, is a rule that can be used when rows are inserted, updated, or deleted to check the affected table after the operation. There are two types of constraints: *unique constraints* check that the new values in a column are unique throughout the table, and *foreign-key constraints* specify how related records should be updated when a record in another table is updated.
- **DataRelation** objects— **DataRelation** objects specify a relationship between parent and child tables, based on a key that both tables share.
- **DataRow** objects— **DataRow** objects correspond to a particular row in a data table. You use the **Item** property to get or set a value in a particular field in the row.

DataColumn objects— **DataColumn** objects correspond to the columns in a table. Each object has a **DataType** property that specifies the kind of data each column contains, such as integers or string values.

Using Basic SQL

You can use Structured Query Language (SQL) in data adapters to configure what those data adapters do.

Using the SELECT Statement

You use the **SELECT** statement to get fields from a table

```
SELECT * FROM Customers
```


This returns a dataset that holds all the records in the **Customers** table. You also can use the **SELECT** statement to select specific fields from a table CustomerID, Address, and City fields of all the records in the **Customers** table:

```
SELECT CustomerID, Address, City FROM Customers
```

Using Where Clauses

```
SELECT * FROM Customers WHERE City = "Boston"
```

You don't have to use an equals sign here; you can test fields using these operators:

- < (less than)
- <= (less than or equal to)
- > (greater than)
- >= (greater than or equal to)
- **BETWEEN**
- **IN**
- **LIKE**

You can use the **BETWEEN** clause to indicate a range of values you will accept.

```
SELECT * FROM Customers WHERE CustomerID BETWEEN  
"H*" AND "I"
```

Note the use of wildcard characters: "H*" and "I*". Using these wildcards lets you specify that you want all the CustomerID values that start with the letter H.

```
SELECT * FROM Customers WHERE City IN ("Boston",  
"York")  
SELECT * FROM Customers WHERE City LIKE "Los*"  
SELECT DISTINCT City FROM Customers  
SELECT * FROM Customers WHERE City NOT IN ("Boston",  
"York") AND Fax IS NOT NULL
```

You can order the records in the dataset using an SQL statement

```
SELECT * FROM Customers ORDER BY CustomerID
```

You can also sort in descending order with the **Desc** keyword:

```
SELECT * FROM Customers ORDER BY CustomerID Desc  
SELECT ContactName AS Name FROM Customers
```

Now in the dataset, the ContactName field will be called Name.

Using Built-in Functions

- **COUNT**— gets a count of records.
- **SUM**— adds values over records.
- **MIN**— finds the minimum value of a set of records.
- **MAX**— finds the maximum value of a set of records.
- **AVG**— finds the average value of a set of records.

```
SELECT COUNT(EmployeeID) AS NumberEmployees,  
AVG(DateOfBirth) AS AverageDateOfBirth, SUM(DateOfBirth)  
AS TotalYears, MIN(DateOfBirth) AS MinDateOfBirth,  
MAX(DateOfBirth) AS MaxDateOfBirth FROM Employees
```

```
SELECT * FROM Customers GROUP BY City  
SELECT * FROM Customers GROUP BY City HAVING City LIKE "Los*"
```

```
DELETE * FROM Customers WHERE City NOT IN ("Boston", "York")
```

You can use the **UPDATE** statement to update a database.

```
UPDATE Customers SET City = "Boston" WHERE City = "York"
```

Joining Tables

```
SELECT Customers.ContactName, Orders.OrderID FROM  
Customers, Orders WHERE Customers.CustomerID =  
Orders.CustomerID
```

Creating a Dataset

To create a dataset using a data adapter, you can use the Data|Generate Dataset menu item or right-click a data adapter and choose the Generate Dataset menu item.

Datasets are disconnected from data sources, and to populate them with data, you must use a data adapter. you can use a data adapter's **Fill** method to fill a dataset:

```
DataSet11.Clear()  
SqlDataAdapter1.Fill(DataSet11)
```

Selecting a Data Provider (SQL Server, MS Jet, and so on)

By default, Visual Basic assumes that you're working with SQL Server when you create a data connection. However, you can change that when you create a connection by informing Visual Data Access Using Data Adapter Controls

Previewing Data from Data Adapters

After you've set up a data adapter, you can get a quick look at the data it provides with the Data Adapter Preview dialog. Just click the Data|Preview Data menu item, or right-click an adapter and select the Preview Data menu item click the Fill Dataset button to display the

data from the adapter.

Examining Dataset Properties

You can take a look at the Dataset properties for the dataset(s) in an application by using the Data|Dataset Properties menu item, or by right-clicking a dataset and selecting the Dataset Properties item.

Connecting to an MS Jet Database

Visual Basic's data access was originally built to work with the Microsoft Jet data access engine, which is the engine in Microsoft Access. Plenty of people still use Jet databases, often because they use MS Access to create them.

```
Private Sub Form1_Load(ByVal sender As System.Object, _  
    ByVal e As System.EventArgs) Handles MyBase.Load  
    dsStudents1.Clear()  
    OleDbDataAdapter1.Fill(dsStudents1)  
End Sub
```

Adding Multiple Tables to a Dataset

A single dataset can contain multiple tables. You can bind two data grids to the same dataset, displaying a different table in each data grid. (After setting the data grids' **DataSource** property to the new dataset)

Using Data Views

Data views are much like read-only mini-datasets; you typically load only a subset of a dataset into a data view. This connects the data grid to the data view, and if we did nothing more, the data grid would pass the entire table on to the data grid. However, you usually use data views to work with just a subset of the data in a table, and you can specify what subset you want using the **RowFilter** property.

```
Private Sub Form1_Load(ByVal sender As System.Object, _  
    ByVal e As System.EventArgs) Handles MyBase.Load  
    DataSet11.Clear()  
    SqlDataAdapter1.Fill(DataSet11)  
End Sub
```

You also can use a data view's **DataViewRowState** property to add rows to a data view depending on their *state*. For example, you can place rows that have been deleted or are new in a data view using this property. Here are the possible state values you can use, and the types of rows they match, from the **DataViewRowState** enumeration:

- **Added**— Added rows.

- **CurrentRows**— Current rows including unchanged, new, and modified rows.
- **Deleted**— Deleted rows.
- **ModifiedCurrent**— The current rows, a modified version of the original data.
- **ModifiedOriginal**— The original rows which have since been modified; the modified rows are available in **ModifiedCurrent**.
- **None**— No rows.
- **OriginalRows**— Original rows, including unchanged and deleted rows.
- **Unchanged**— Unchanged rows.

For example, here's how we make sure that only rows that have been added to a table appear in a data view:

```
Private Sub Form1_Load(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles MyBase.Load
    DataSet11.Clear()
    SqlDataAdapter1.Fill(DataSet11)
    DataView1.RowStateFilter = DataViewRowState.Added
End sub
```

Simple Binding

Simple binding lets you to display *one* data element, such as a field's value from a data table, in a control. In Visual Basic .NET, you can simple-bind *any* property of a control to a data value.

For example, to bind a text box's **Text** property to the rollnumber field in the **student** table from the database in a dataset, **DataSet11**, you select the text box and expand its **(DataBindings)** property in the Properties window. You can select a field in a dataset to bind to just by clicking a property and selecting a table from the drop-down list that appears.

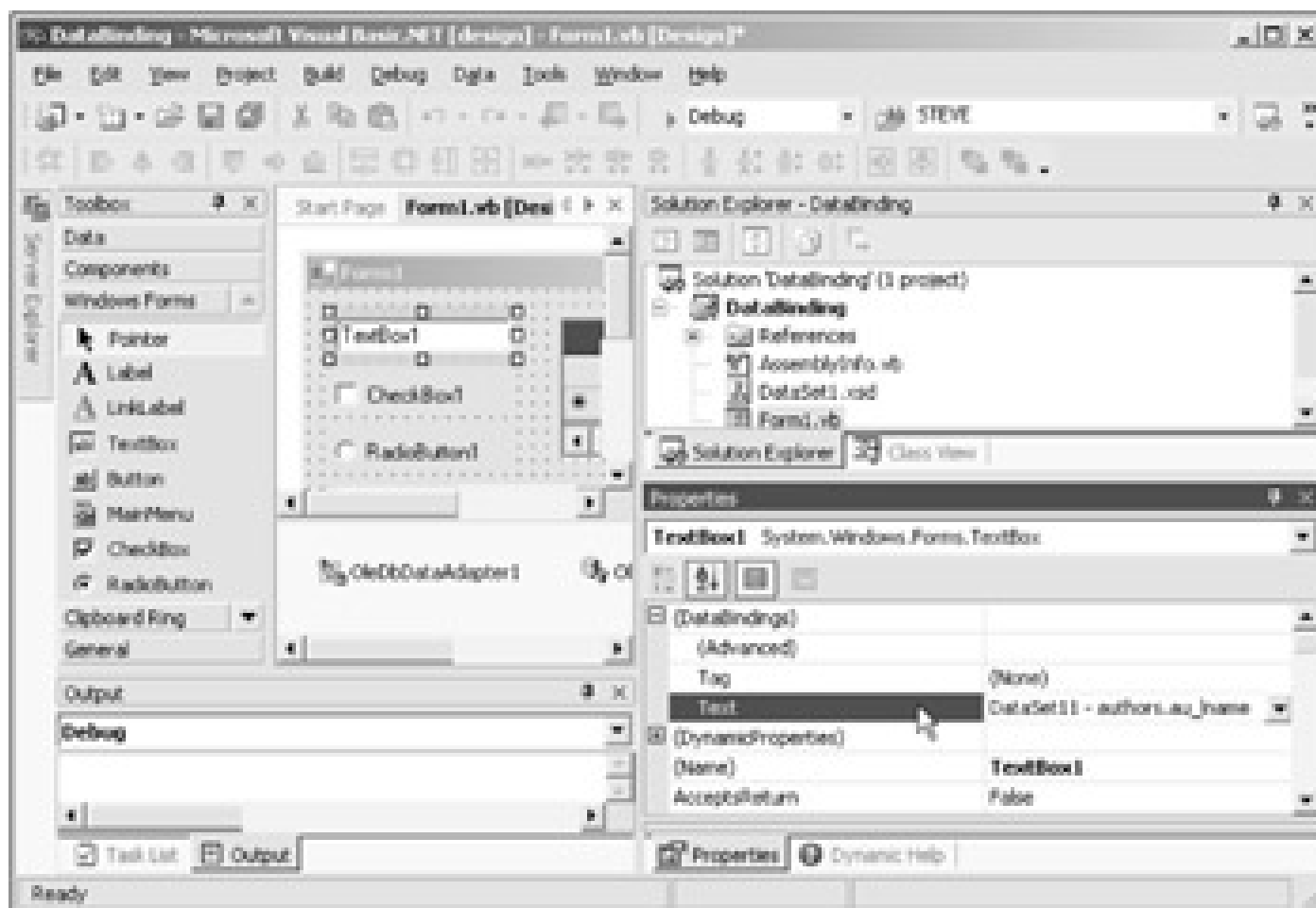


Figure 21.1: Binding a text box to a data table.

As mentioned, you can bind to any property of a control. To do that, click the ellipsis button that appears when you click the **(Advanced)** entry in the **(DataBindings)** property, opening the Advanced Data Binding dialog you see in Figure 21.2.

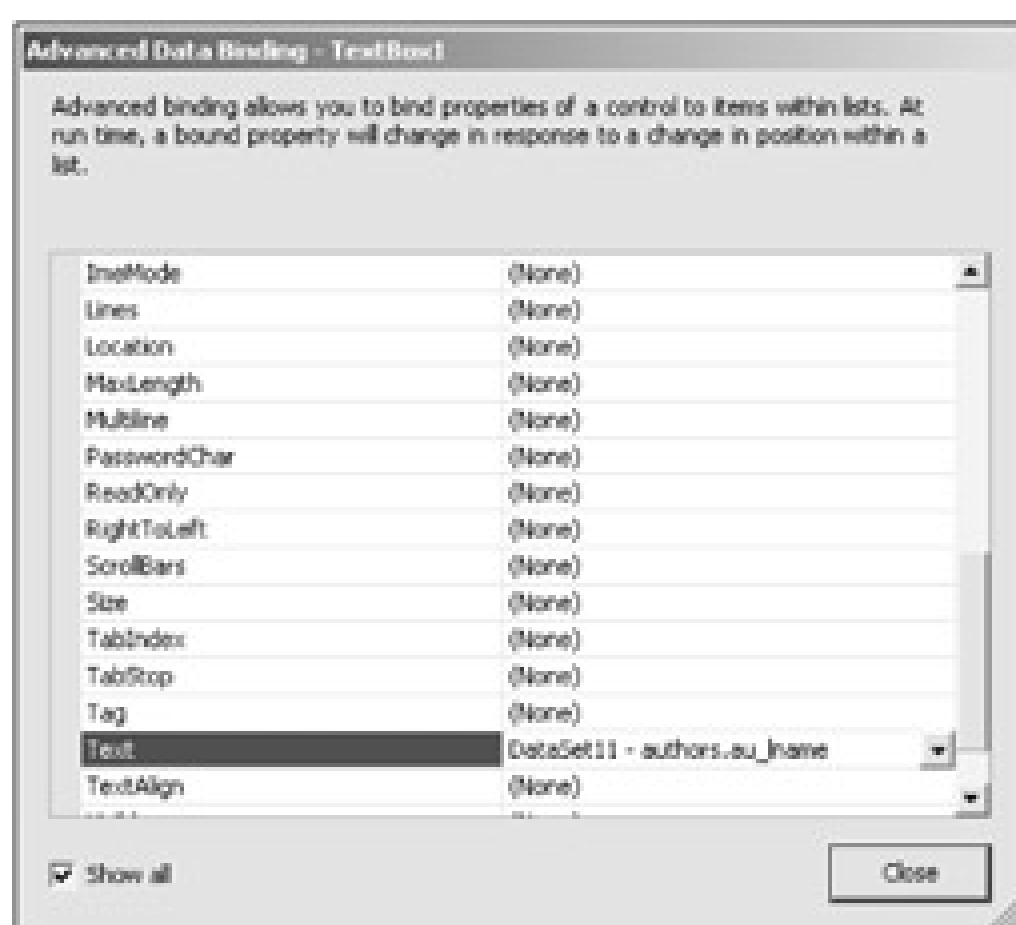


Figure 21.2: The Advanced Data Binding dialog.

Using the Advanced Data Binding dialog, you can bind any property of a control to a data source.

Note that because simple-bound controls show only one data element at a time (such as the current rollover), it's usual to include navigation controls in a Windows form with simple-bound controls;

The navigation controls will let the user move from record to record just by clicking buttons, and all the data in the bound controls will be updated automatically to match.

You also can perform simple binding in code, using a control's **DataBindings** property. For example, in code, we could bind the text box to the same rollnumber field that we just bound it to at design time. Using the collection's **Add** method, you pass this method the property to bind, the data source to use, and the specific field you want to bind:

```
TextBox1.DataBindings.Add("Text", DataSet11, "rollnumber ")
```

The **Add** method is overloaded so that you can pass it a **Binding** object directly.

```
DateTimePicker1.DataBindings.Add(New Binding("Value",  
DataSet11, "customers.DeliveryDate"))
```

You can even bind one control to another in code this way. Here binding the **Text** property of one text box to another, which means that if you change the text in the source text box, the text in the bound text box will change immediately to match:

```
TextBox2.DataBindings.Add("Text", TextBox1, "Text")
```

Table 21.1: Noteworthy public properties of *ControlBindingsCollection* objects.

Property	Means
Control	Gets the collection's associated control.
Count	Gets the number of items in the collection.
Item	Gets a binding.

Table 21.2: Noteworthy public methods of *ControlBindingsCollection* objects.

Method	Means
Add	Adds a binding.
Clear	Clears the collection.
Remove	Deletes a binding.
RemoveAt	Deletes a binding for the specified item.

Table 21.3: Noteworthy public events of *ControlBindingsCollection* objects.

Event	Means
CollectionChanged	Occurs when collection changes.

Example:
DateTimePicker1.DataBindings.Add (New Binding("Value", DataSet11, "customers.DeliveryDate"))

The **Binding** class also lets you format values for display with the **Format** event and to read formatted values with the **Parse** event.

Table 21.4: Noteworthy public properties of <i>Binding</i> objects.	
Property	Means
BindingMemberInfo	Gets an object that contains binding information.
Control	Gets the associated control.
DataSource	Gets the binding's data source.
IsBinding	Gets whether the binding is active.
PropertyName	Gets/sets the control's data-bound property.

Table 21.5: Noteworthy public events of *Binding* objects.

Table 21.5: Noteworthy public events of <i>Binding</i> objects.	
Event	Means
Format	Occurs when a property is bound to a data value.
Parse	Occurs when a data-bound control's value changes.

DataGrid

Here is the inheritance hierarchy of this class:



Noteworthy public properties of <i>DataGrid</i> objects.	
Property	Means
AllowNavigation	Gets/sets if navigation is possible.

AllowSorting	Gets/sets if the grid can be sorted when the user clicks a column header.
AlternatingBackColor	Gets/sets the background color of alternating rows.
BackColor	Gets/sets the background color of the grid.
BackgroundColor	Gets/sets the color of the non-data part of the data grid.
BorderStyle	Gets/sets the grid's style of border.
CaptionBackColor	Gets/sets the caption's background color.
CaptionFont	Gets/sets the caption's font.
CaptionForeColor	Gets/sets the caption's foreground color.
CaptionText	Gets/sets the caption's text.
CaptionVisible	Gets/sets if the caption is visible.
ColumnHeadersVisible	Gets/sets if the parent rows of a table are visible.
CurrentCell	Gets/sets which cell has the focus.
CurrentRowIndex	Gets/sets the index of the selected row.
DataMember	Gets/sets the table or list of data the data grid should display.
DataSource	Gets/sets the data grid's data source, such as a dataset.
FirstVisibleColumn	Gets the index of the first column visible in the grid.
FlatMode	Gets/sets if the grid is shown flat.
ForeColor	Gets/sets the foreground color.
GridLineColor	Gets/sets the color of grid lines.
GridLineStyle	Gets/sets the grid line style.
HeaderBackColor	Gets/sets the background color of headers.
HeaderFont	Gets/sets the font used for headers.
HeaderForeColor	Gets/sets the foreground color of headers.
Item	Gets/sets the value in a particular cell.
LinkColor	Gets/sets the color of links to child tables.
LinkHoverColor	Gets/sets the color of links when the mouse moves over it.
ParentRowsBackC	Gets/sets the background color of parent rows.

Color	
ParentRowsForeColor	Gets/sets the foreground color of parent rows.
ParentRowsLabelStyle	Gets/sets the style for parent row labels.
ParentRowsVisible	Gets/sets if parent rows are visible.
PreferredColumnWidth	Gets/sets the width of the grid columns (measured in pixels).
PreferredRowHeight	Gets/sets the preferred row height.
ReadOnly	Gets/sets if the grid is read-only.
RowHeadersVisible	Gets/sets if row headers are visible.
RowHeaderWidth	Gets/sets the width of row headers.
SelectionBackColor	Gets/sets selected cell's background color.
SelectionForeColor	Gets/sets selected cell's foreground color.
TableStyles	Gets the table styles in the data grid.
VisibleColumnCount	Gets the number of visible columns.
VisibleRowCount	Gets the number of visible rows.

Noteworthy public methods of *DataGrid* objects.

Noteworthy public methods of <i>DataGrid</i> objects.	
Method	Means
BeginEdit	Allows editing.
Collapse	Collapses child table relations.
EndEdit	Ends editing operations.
Expand	Displays child relations.
GetCellBounds	Gets the Rectangle object that specifies a cell.
GetCurrentCellBounds	Gets a Rectangle object that specifies the selected cell.
HitTest	Coordinates mouse position with points in the data grid.

IsExpanded	Gets whether a row is expanded or collapsed.
IsSelected	Gets whether a row is selected.
NavigateBack	Navigates to the previous table that was shown in the grid.
NavigateTo	Navigates to a specific table.
Select	Makes a selection.
SetDataBinding	Sets both the DataSource and DataMember properties. Used at run time.
UnSelect	Unselects a row.

Noteworthy public events of *DataGrid* objects.

Noteworthy public events of <i>DataGrid</i> objects.	
Event	Means
AllowNavigationChanged	Occurs when the AllowNavigation property changes.
CurrentCellChanged	Occurs when the CurrentCell property changes.
DataSourceChanged	Occurs when the DataSource property value changes
FlatModeChanged	Occurs when the FlatMode changes.
Navigate	Occurs when the user navigates to a new table.
ParentRowsVisibleChanged	Occurs when the ParentRowsVisible property value changes.
ReadOnlyChanged	Occurs when the ReadOnly property value changes.
Scroll	Occurs when the data grid is scrolled.

Complex Binding

Simple data binding binds to one data item at a time, such as a name displayed in a text box, but complex data binding allows a control to bind to more than one data element, such as more than one record in a database, at the same time. When we bound an entire data table to a data grid. Instead of displaying just one data item at a time, the data grid displayed the entire table at once, including all the fields and field data. Most controls support only simple data binding but some, such as data grids and list boxes, support complex data binding.

Complex data binding revolves around these properties:

- **DataSource**— The data source, typically a dataset such as **DataSet11**.
- **DataMember**— The data member you want to work with in the data source, typically a table in a dataset such as the **authors** table in the pubs database. Data grids use this property to determine which table they should display.
- **DisplayMember**— The field you want a control to display, such as the author's last name, au_lname. List boxes use the **DisplayMember** and **ValueMember** properties instead of a **DataMember** property.
- **ValueMember**— The field you want the control to return in properties like **SelectedValue**, such as au_id. List boxes use the **DisplayMember** and **ValueMember** properties instead of a **DataMember** property.

For example, to bind a dataset, **DataSet11**, to a data grid in code, showing the **authors** table:

```
DataGrid1.DataSource = DataSet11
DataGrid1.DataMember = "authors"
```

You also can use the built-in data grid method named **SetDataBinding** for this (data grids are the only controls that have this method):

```
DataGrid1.SetDataBinding(dsDataSet, "authors")
```

to bind the **dsDataSet** dataset to a list box using the **DisplayMember** property, using the au_lname field in the **authors** table; note the syntax used to specify a field in a table: authors.au_lname:

```
ListBox1.DataSource = dsDataSet
ListBox1.DisplayMember = "authors.au_lname"
```

Binding Text Boxes

Text boxes are simple data-binding controls. Here are the commonly bound properties displayed when you expand the **(DataBindings)** property:

- **Tag**
- **Text**

You perform simple data binding by adding **Binding** objects to a **ControlBindingsCollection**. The **ControlBindingsCollection** contains standard collection methods such as **Add**, **Clear**, and **Remove**


```
DateTimePicker1.DataBindings.Add(New Binding("Value",  
DataSet11, "customers.DeliveryDate"))
```

And you also can use another overloaded form of the **Add** method to create a simple data binding, like this:

```
TextBox1.DataBindings.Add("Text",DataSet11,"authors.au_lname")
```

Binding Checkboxes

Checkboxes are simple data-binding controls. Here are the commonly bound properties displayed when you expand the **(DataBindings)** property:

- **CheckAlign**
- **Checked**
- **CheckState**
- **Tag**
- **Text**

Binding List Boxes

List boxes are complex data-binding controls. Here are the properties you use to bind this control to a data source

- **DataSource**
- **DisplayMember**
- **ValueMember**

Binding Combo Boxes

Combo boxes are complex data-binding controls. Here are the properties you use to bind this control to a data source .

- **DataSource**
- **DisplayMember**
- **ValueMember**

Binding Checked List Boxes

Checked list boxes are complex data-binding controls. Here are the properties

you use to bind this control to a data source .

- **DataSource**
- **DisplayMember**
- **ValueMember**

Binding Data Grids

```
DataGrid1.SetDataBinding(dsDataSet, "authors")
```

You can use these data sources with the data grid's **DataSource** property:

- **DataTable** objects
- **DataView** objects
- **DataSet** objects
- **DataViewManager** objects
- single dimension arrays

To determine which cell was selected by the user, use the **CurrentCell** property. You can change the value of any cell using the **Item** property, which can take either the row or column indexes of the cell. And you can use the **CurrentCell Changed** event to determine when the user selects another cell.

Using the *BindingContext* Class

You use the **BindingContext** class to access the data bindings in a control, including a form. Here is the inheritance hierarchy of this class:



Each object that inherits from the **Control** class can have a single **Binding Context** object. Using this object gives you access to the data bindings in a form which allows you to set the current record displayed in simple-bound controls, using the **Position** property.

Table 21.9: Noteworthy public properties of <i>BindingContext</i> objects.	
Property	Means

Item	Gets a particular binding.
------	----------------------------

Table 21.10: Noteworthy public methods of *BindingContext* objects.

Table 21.10: Noteworthy public methods of <i>BindingContext</i> objects.	
Method	Means
Contains	Gets whether the BindingContext contains the specified item.

Navigating in Datasets

You usually add navigation controls, such as the buttons at the bottom of the DataBinding. Let the user move from record to record. When the user clicks the > button, for example, the bound data from the next field is displayed in the text box, so the name displayed in the text box changes from the first author's last name to the last name of the next author. The >> button moves to the last record, as you'd expect; the << button moves to the first record, and so on.

Displaying the Current Location

When the DataBinding example first loads, it fills its dataset from its data adapter. It also displays the current position in the dataset's binding context for the **authors** table by using the form's **Binding Context** property's **Position** and **Count** members, like this:

```
Private Sub Form1_Load(ByVal sender As system.Object,
ByVal e As System.EventArgs) Handles MyBase.Load
    DataSet11.Clear()
    OleDbDataAdapter1.Fill(DataSet11)
    Label1.Text = (Me.BindingContext(DataSet11, "authors")
Position + 1).ToString & " of " &
Me.BindingContext(DataSet11, "authors").Count.ToString
End Sub
```

Moving to the Next Record

when the user clicks the > arrow, just increment the **Position** property of the

binding context for the **authors** table, and then display the new location in the label between the navigation buttons:

```
Private Sub Button3_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button3.Click
    Me.BindingContext(DataSet11, "authors").Position = _
        (Me.BindingContext(DataSet11, "authors").Position
+ 1)
    Label1.Text = (((Me.BindingContext(DataSet11,
"authors").Position + 1).ToString & " of ") &
Me.BindingContext(DataSet11, "authors").Count.ToString)
End Sub
```

Note that if you try to move beyond the end of the record set, the **Position** property isn't incremented.

Moving to the Previous Record

In the same way, when the user clicks the < button to move to the previous record, the code simply decrements the **Position** value:

```
Private Sub Button2_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button2.Click
    Me.BindingContext(DataSet11, "authors").Position = _
        (Me.BindingContext(DataSet11, "authors").Position - 1)
    Label1.Text=(((Me.BindingContext(DataSet11, "authors").Position + 1).ToString & " of ") &
Me.BindingContext(DataSet11, "authors").Count.ToString)
End Sub
```

Moving to the First Record

Moving to the first record in the binding context for the **authors** table is easy; you just set the **Position** property to 0:

```
Private Sub Button4_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button4.Click
    Me.BindingContext(DataSet11, "authors").Position = 0
    Label1.Text = (((Me.BindingContext(DataSet11,
"authors").Position+ 1).ToString & " of ") &
Me.BindingContext(DataSet11, "authors").Count.ToString)
End Sub
```

Moving to the Last Record

Moving to the last record is also easy, because we know that the **Count** property returns the total number of records in the table:

```
Private Sub Button5_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button5.Click
```

```

        Me.BindingContext(DataSet11, "authors").Position = _
            Me.BindingContext(DataSet11, "authors").Count - 1
        Label1.Text = (((Me.BindingContext(DataSet11,
            "authors").Position + 1).ToString & " of ") &
            Me.BindingContext(DataSet11, "authors").Count.ToString)
    End Sub

```

Using the DisplayMember and ValueMember Properties

As we know, list boxes use **DisplayMember** and **ValueMember** properties to bind to a specific data field. Bound the checked list box's **DisplayMember** property to the authors.au_lname field, and the **ValueMember** property to the authors.au_id field. That means the program will show the author's last name in the checked list box, but when the user clicks an author in that control, I'll use the control's **SelectedValue** property to get the actual author's ID value and display it:

```

Private Sub CheckedListBox1_SelectedIndexChanged(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles CheckedListBox1.SelectedIndexChanged
    TextBox2.Text = "Selected ID: " &
checkedListBox1.SelectedValue
End Sub

```

Adding Records to a Dataset

Adding records to a dataset is easy, as you can see in the code the data form uses for this purpose— all you have to do is to use the **BindingContext** object's **AddNew** method:

```

Private Sub btnAdd_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles btnAdd.Click
    Try
        'Clear out the current edits
        Me.BindingContext(objdsDataSet1,
            "authors").EndCurrentEdit()
        Me.BindingContext(objdsDataSet1, "authors").AddNew()
    Catch eEndEdit As System.Exception

        System.Windows.Forms.MessageBox.Show(eEndEdit.Message)
    End Try
    Me.objdsDataSet1_PositionChanged()
End Sub

```


Sub program to display records

```
Private Sub objdsDataSet1_PositionChanged()  
    Me.lblNavLocation.Text = _  
        (((Me.BindingContext(objdsDataSet1, _  
            "authors").Position + 1).ToString + " of ") _  
        + Me.BindingContext(objdsDataSet1, _  
            "authors").Count.ToString)  
End Sub
```

Deleting Records from a Dataset

To delete a record from the dataset, all you have to do is to use the **RemoveAt** method of the binding context:

```
Private Sub btnDelete_Click(ByVal sender As System.Object,  
    ByVal e As System.EventArgs) Handles btnDelete.Click  
    If (Me.BindingContext(objdsDataSet1, "authors").Count > 0)  
Then  
    Me.BindingContext(objdsDataSet1, _  
        "authors").RemoveAt(Me.BindingContext(objdsDataSet1, _  
            "authors").Position)  
    Me.objdsDataSet1_PositionChanged()  
End If  
End Sub
```

Canceling a Dataset Edit

When you begin to change the data in the controls bound to a dataset, that in turn starts an edit operation in the dataset. The next time the Update button is clicked, the edited records will be sent to the database itself. If you want to cancel the current edit operation, you can click the Cancel button, which uses the **CancelCurrentEdit** method:

```
Private Sub btnCancel_Click(ByVal sender As System.Object,  
    ByVal e As System.EventArgs) Handles btnCancel.Click  
    Me.BindingContext(objdsDataSet1, _  
        "authors").CancelCurrentEdit()  
    Me.objdsDataSet1_PositionChanged()  
End Sub
```

Updating the Underlying Data Store

Working with the dataset is fairly easy, but when it comes to sending the changes back to the database itself, things get more complex, as you can see by taking a look at the code for the Update button in the data form. This code starts by calling a Sub procedure named **UpdateDataSet**:

```

Private Sub btnUpdate_Click(ByVal sender As System.Object,
    ByVal e As System.EventArgs) Handles btnUpdate.Click
    Try
        Me.UpdateDataSet()
    Catch eUpdate As System.Exception

        System.Windows.Forms.MessageBox.Show(eUpdate.Message)
    End Try
    Me.objdsDataSet1_PositionChanged()
End Sub

```

In the **UpdateDataSet** Sub procedure, we want to send the changes that have been made back to the database, so the code starts by creating a new dataset that holds only the changed records, using the current dataset's **GetChanges** method. Then it sends that new dataset to a Sub procedure called **UpdateDataSource** to update the data source. After the call to **UpdateDataSource**, the code calls the **AcceptChanges** method of the main dataset so the current edit operation ends and the changed records are no longer considered "changed" records, but simply normal records:

```

Public Sub UpdateDataSet()
    'Create a new dataset to hold the changes that have been
    made to the main dataset.
    Dim objDataSetChanges As DataForm.dsDataSet1 = _
        New DataForm.dsDataSet1()
    'Stop any current edits.
    Me.BindingContext(objdsDataSet1,
    "authors").EndCurrentEdit()
    'Get the changes that have been made to the main dataset.
    objDataSetChanges = CType(objdsDataSet1.GetChanges, _
        DataForm.dsDataSet1)
    'Check to see if any changes have been made.
    If (Not (objDataSetChanges) Is Nothing) Then
        Try
            'There are changes that need to be made, so attempt
            'to update the datasource by
            'calling the update method and passing the dataset
            and any parameters.
            Me.UpdateDataSource(objDataSetChanges)
            objdsDataSet1.Merge(objDataSetChanges)
            objdsDataSet1.AcceptChanges()
        Catch eUpdate As System.Exception
            'Add your error handling code here.
            Throw eUpdate
        End Try
        'Add your code to check the returned dataset

```

```

        'for any errors that may have been
        'pushed into the row object's error.
    End If
End Sub

```

To actually pass the changes back to the database, the code uses the **Update** data adapter method. Here's how that works in the **UpdateDataSource** Sub procedure:

```

Public Sub UpdateDataSource(ByVal ChangedRows As
DataForm.dsDataSet1)
    Try
        'The data source only needs to be updated if there are
        'changes pending.
        If (Not (ChangedRows) Is Nothing) Then
            'Open the connection.
            Me.OleDbConnection1.Open()
            'Attempt to update the data source.
            OleDbDataAdapter1.Update(ChangedRows)
        End If
        Catch updateException As System.Exception
            'Add your error handling code here.
            Throw updateException
        Finally
            'Close the connection whether or not the exception
            was thrown.
            Me.OleDbConnection1.Close()
        End Try
    End Sub

```

How do we place a value into the SQL parameter corresponding to the state field at run time?

```

Private Sub Button1_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button1.Click
    OleDbDataAdapter2.SelectCommand.Parameters("state").Value =
        ComboBox1.Text
    DataSet12.Clear()
    OleDbDataAdapter2.Fill(DataSet12)
    ShowPosition()
End Sub

```

Using Master/Detail Relationships and Data Relation Objects

We can use a single SQL statement to join the data from two related tables (the **authors** and **titleauthor** tables from the pubs database, which are related using the au_id field). However, you don't have to merge the data from two tables into one dataset to work with related tables; you can keep the two tables separate, and use a data relation object to relate them. For example, you might want to set up a master/detail relationship, also called a parent/child relationship, between two tables

The new dataset is just a data container, like any dataset, and that means it doesn't know anything about the relationship between the tables. In fact, both tables share a common field, pub_id, the publisher ID, which is a key into both tables.

We'll also need code to load the two tables into the dataset from the two data adapters, so add this code to the **Form1_Load** event:

```
Private Sub Form1_Load(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles MyBase.Load
    SqlDataAdapter1.Fill(DataSet11)
    SqlDataAdapter2.Fill(DataSet11)
End Sub
```

Using the *ErrorProvider* Class

The **ErrorProvider** class gives you a user interface when you want to indicate that there's an error associated with a control in a form; an icon will appear next to the control and an error message in the icon's tool tip. Here is the inheritance hierarchy for this class:

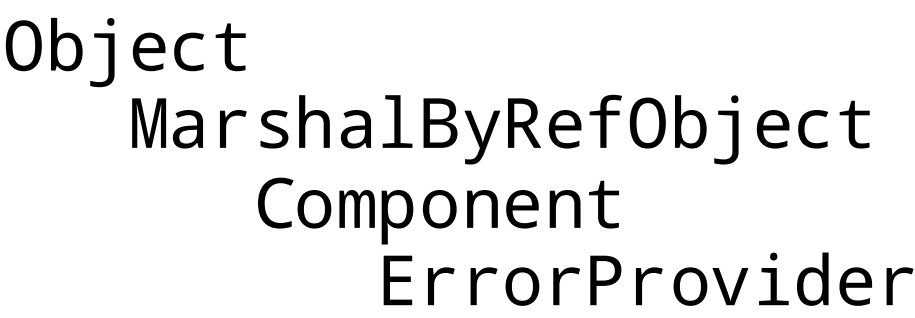


Table 21.11: Noteworthy public properties of <i>ErrorProvider</i> objects.	
Property	Means
BlinkRate	Gets/sets the error icon blink rate.
BlinkStyle	Gets/sets if the error icon flashes.
ContainerControl	Gets/sets the parent control for this error provider.

Table 21.11: Noteworthy public properties of *ErrorProvider* objects.

Property	Means
DataMember	Gets/sets the data table to watch.
DataSource	Gets/sets dataset to watch.
Icon	Gets/sets the icon to be displayed next to a control when you've assigned a non-empty string to the error provider.

Table 21.12: Noteworthy public methods of *ErrorProvider* objects.

Method	Means
BindToDataAndErrors	Sets both the DataSource and DataMember properties at run time.
GetError	Returns the error description string.
GetIconAlignment	Gets the position of the error icon with respect to the control.
GetIconPadding	Gets the space to leave next to the icon.
SetError	Sets the error text.
SetIconAlignment	Sets the position of the error icon with respect to the control.
SetIconPadding	Sets the space to leave next to the icon.
UpdateBinding	Lets you update the data bindings and the text in the error provider.

Performing Data Validation in Controls

Now that we're creating data-entry forms, it's useful to know that you can *validate* the data the user enters into controls. If a control contains data that you think invalid, you can use an error provider. Here are the control events and properties you use in data validation:

- **Validating Event**— Occurs when the control is validating, which happens when the control loses the focus (if the control's **CausesValidation** property is **True**). Place code to check the control's data here, and throw an exception if there's a problem. You can set an error provider's message here.
- **Validated Event**— Occurs when the control is done validating. This event occurs if no exception was thrown in the **Validating** event. You can clear an error provider's message here.
- **CausesValidation Property**— **True** if entering the control causes validation to be performed on other controls requiring validation when this control gets the focus; otherwise, **False**. The default is **True**.

example

```
Private Sub TextBox1_Validating(ByVal sender As Object, _
    ByVal e As System.ComponentModel.CancelEventArgs)
Handles _
    TextBox1.Validating
    If TextBox1.Text.Length = 0 Then
        ErrorProvider1.SetError(TextBox1, "Please enter a
value")
        Throw New Exception("You must enter a value.")
    End If
End Sub
```

If the text box data did not validate, the icon and tool tip appear, on the other hand, if the data does validate properly, no exception is thrown by the code in the **Validating** event, and the **Validated** event for the control occurs.

```
Private Sub TextBox1_Validated(ByVal sender As Object, _
    ByVal e As System.EventArgs) Handles TextBox1.Validated
    ErrorProvider1.SetError(TextBox1, "")
End Sub
```
